

Managed Scenario Save Sequence(Additional Save and Unmanaged Save).

The managed save sequence is a key concept in the RAP (Restful Application Programming) framework used to handle persistence operations for entity instances in ABAP. It automates the process of saving changes made to an entity while allowing customization when needed.

Key Components of the Managed Save Sequence

1. Standard Behavior

- Automatically saves changes made to entity instances in the database.
- Ensures consistent data persistence without requiring additional coding.

2. Steps in the Sequence

- Data Validation: Validates the modified instance to ensure data integrity.
- Data Modification: Updates or inserts the instance data into the database.
- Lifecycle Management: Manages the lifecycle events triggered during the save process.
- Post-Save Actions: Completes any final actions after the data persisted.

3. Customizing with Extra Save

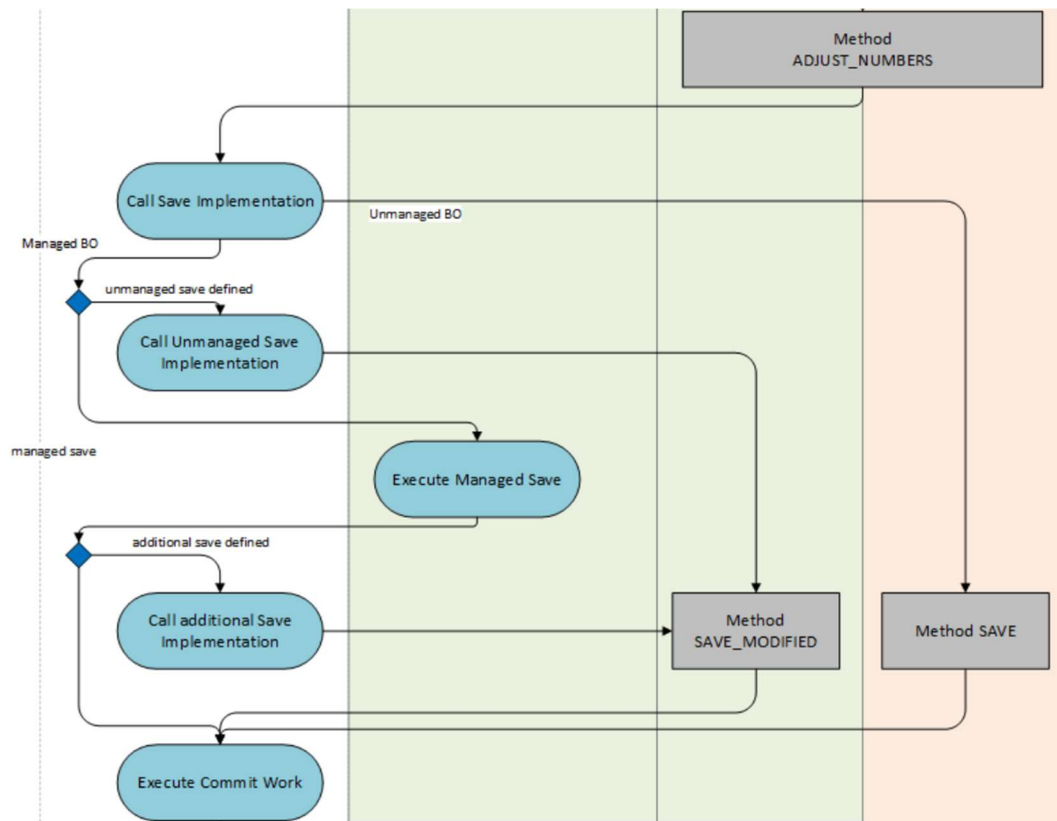
- The extra save feature allows adding additional logic to the save process, such as logging changes or triggering external services.
- Developers can implement additional steps using the `save_modified` method in a saver class.

4. Change Tracking

- Logs changes made to an instance, such as modified fields or newly added data.
- These changes can be written to a log table for auditing or debugging purposes.

Tuesday, December 30, 2024

By: Devi Prasad Mishra



1. Additional Save

The RAP framework supports adding extra functionality to the managed save sequence via the extra save feature, allowing reuse of services like change documents and application logs. By default, managed saves handle this, but additional steps can be included using the saver class and the save modified method. When an entity instance is updated, changes are saved, including updated fields, data, and completed operations. In this scenario, modified data is recorded in a log table.

Action: Create the log table to store the changes.

```
1 @EndUserText.label : 'log table'
2 @AbapCatalog.enhancement.category : #NOT_EXTENSIBLE
3 @AbapCatalog.tableCategory : #TRANSPARENT
4 @AbapCatalog.deliveryClass : #A
5 @AbapCatalog.dataMaintenance : #RESTRICTED
6 define table zlog_tab_upd {
7
8     key changeid      : abap.raw(16) not null;
9     salerid           : zsale;
10    materialid        : zmatno;
11    imageid           : zimgid;
12    change_operation   : abap.char(10);
13    changed_field      : abap.char(40);
14    changed_value      : abap.char(40);
15    created_at         : timestamp;
16
17 }
```

Tuesday, December 30, 2024

By: Devi Prasad Mishra

To enable additional saving in a managed scenario:

- a) **Behavior Definition:** Define the additional save for the entity.
- b) **Implementation Class:** Create a local class and implement the save_modified method.

This setup ensures the modified data is handled appropriately.

```
managed implementation in class zbp_i_saler unique;
strict ( 2 );

•define behavior for zi_saler
implementation in class zbp_i_saler unique
persistent table zsalers table
with additional save
lock master
authorization master ( instance )
early numbering
{

    create;
    update;
    delete;
    field ( readonly ) SalerId;
    field ( mandatory ) name;
    association _material { create; }
```

When a new instance is created, additional data is stored in the entity's database and log table.

Steps:

- 1) **Define Saver Class:** Create a saver class in the behavior definition.
- 2) **Behavior Pool:** Implement required methods in the behavior pool.
- 3) **Save_Modified Method:** Implement the save_modified method in the saver class to handle the additional save logic.

This ensures proper handling of additional data during entity creation.

```
1•CLASS lhs_zi_saler DEFINITION INHERITING FROM cl_abap_behavior_saver.
2  PROTECTED SECTION.
3  METHODS : save_modified REDEFINITION.
4  ENDCLASS.
5
```

```
6•CLASS lhs_zi_saler IMPLEMENTATION.
7
8•  METHOD save_modified.
9    DATA : lt_log TYPE STANDARD TABLE OF zlog_tab_upd.
10   DATA : lt_log_c TYPE STANDARD TABLE OF zlog_tab_upd.
11   *-----
12   * update the log when we create a seller information
13   *-----
14
15   IF create-zi_saler IS NOT INITIAL.
16     lt_log = CORRESPONDING #( create-zi_saler ).
17
18   LOOP AT lt_log ASSIGNING FIELD-SYMBOL(<lfs_log>).
19     GET TIME STAMP FIELD <lfs_log>-created_at.
20
21     READ TABLE create-zi_saler ASSIGNING FIELD-SYMBOL(<lfs_create>) WITH TABLE KEY entity
22                                     COMPONENTS SalerId = <lfs_log>-salerid.
23
24   IF sy-subrc IS INITIAL.
25     IF Sales is created
26       IF <lfs_create>-%control-SalerId = cl_abap_behv=>flag_changed.
27         <lfs_log>-changed_field = 'Saler ID'.
28         <lfs_log>-change_operation = 'Created'.
29         <lfs_log>-changed_value = <lfs_create>-SalerId.
30         TRY.
31           <lfs_log>-changeid = cl_system_uuid=>create_uuid_x16_static( ).
32           CATCH cx_uuid_error.
33         ENDTRY.
34         APPEND <lfs_log> TO lt_log_c.
35       ENDIF.
36     UNASSIGN <lfs_create>.
37   ENDLOOP.
38   UNASSIGN <lfs_log>.
39   ENDIF.
```

Tuesday, December 30, 2024

By: Devi Prasad Mishra

```
40 *-----
41 * update the log when we Delete a seller information
42 *-----
43 IF delete-zi_saler IS NOT INITIAL.
44   lt_log = CORRESPONDING #( delete-zi_saler ).
45
46   LOOP AT lt_log ASSIGNING <lfs_log>.
47     <lfs_log>-change_operation = 'Deletion'.
48     GET TIME STAMP FIELD <lfs_log>-created_at.
49
50   *   If Sales is Deleted
51     <lfs_log>-changed_field = 'Saler ID'.
52     <lfs_log>-changed_value = <lfs_log>-SalerId.
53   TRY.
54     <lfs_log>-changeid = cl_system_uuid=>create_uuid_x16_static( ).
55   CATCH cx_uuid_error.
56   ENDTRY.
57   APPEND <lfs_log> TO lt_log_c.
58
59   ENLOOP.
60   UNASSIGN : <lfs_log>.
61 ENDIF.
```

```
62 *-----
63 * update the log when we Update a seller information
64 *-----
65 IF update-zi_saler IS NOT INITIAL.
66   lt_log = CORRESPONDING #( update-zi_saler ).
67
68   LOOP AT update-zi_saler ASSIGNING FIELD-SYMBOL(<lfs_update>).
69     ASSIGN lt_log[ salerid = <lfs_update>-SalerId ] TO <lfs_log>.
70
71     <lfs_log>-change_operation = 'Updated' ##NO_TEXT.
72     GET TIME STAMP FIELD <lfs_log>-created_at.
73
74   *   If Name of seller is Updated
75   IF <lfs_update>-%control-Name = if_abap_behv=>mk-on.
76     <lfs_log>-changed_field = 'Name'.
77     <lfs_log>-changed_value = <lfs_update>-Name.
78   TRY.
79     <lfs_log>-changeid = cl_system_uuid=>create_uuid_x16_static( ).
80   CATCH cx_uuid_error.
81   ENDTRY.
82   APPEND <lfs_log> TO lt_log_c.
83   ENDIF.
84
85   *   If Address of seller is Updated
86   IF <lfs_update>-%control-Address = if_abap_behv=>mk-on.
87     <lfs_log>-changed_field = 'Address'.
88     <lfs_log>-changed_value = <lfs_update>-Address.
89   TRY.
90     <lfs_log>-changeid = cl_system_uuid=>create_uuid_x16_static( ).
91   CATCH cx_uuid_error.
92   ENDTRY.
93   APPEND <lfs_log> TO lt_log_c.
94   ENDIF.
95
96   ENLOOP.
97   UNASSIGN : <lfs_log>, <lfs_update>.
98   ENDIF.
99   *   insert the log data to the table
100  INSERT zlog_tab_upd FROM TABLE @lt_log_C.
101
```

Tuesday, December 30, 2024
By: Devi Prasad Mishra

For creating an instantiation operation.

Unnamed Object

Saler Information

Saler Id:

Name: *

Address:

—

Pawan Kumar

Dhulikhel, Nepal

4153

Edit

Delete

Ctrl+E

Saler Information

Materials Information

Saler Id:

Name:

Address:

4153

Pawan Kumar

Dhulikhel, Nepal

Materials Information

Standard

Search

Q

Available

Not Available

Create

Delete

☐ Material ids

Material

Material Type

UnitField

Quantity

Currencycode

Price

Status

TotalPrice

No items available.

In log table.

CE389E4417721...	4149	Created	Saler ID	4149	20241229115538...
CE389E4417721...	4149	Updated	Name	Devi Prasad	20241230174056...
EA726B4CB5201...	4153	Created	Saler ID	4153	20241231084619...

For Edit.

4153

Delete

Saler Information

Materials Information

Saler Id:

Name: *

Address:

4153

Pawan Mishra

Dhulikhel, Nepal

In log table

CE389E4417721...	4149	Created	Saler ID	4149	20241229115538...
CE389E4417721...	4149	Updated	Name	Devi Prasad	20241230174056...
EA726B4CB5201...	4153	Created	Saler ID	4153	20241231084619...
EA726B4CB5201...	4153	Updated	Name	Pawan Mishra	20241231085016...

For Delete.

4153

Edit

Delete

Ctrl+E

Saler Information

Materials Information

Saler Id:

Name:

Address:

4153

Pawan Mishra

Dhulikhel, Nepal

Tuesday, December 30, 2024

By: Devi Prasad Mishra

In log table

CE3B9E4417721...	4149	Created	Saler ID	4149	20241229115538...
CE3B9E4417721...	4149	Updated	Name	Devi Prasad	20241230174056...
EA726B4CB5201...	4153	Created	Saler ID	4153	20241231084619...
EA726B4CB5201...	4153	Updated	Name	Pawan Mishra	20241231085016...
EA726B4CB5201...	4153	Deletion	Saler ID	4153	20241231085212...

2. Unmanaged Save

Unmanaged save is ideal when the default save logic in the RAP framework (managed save) cannot handle specific business requirements. It gives developers full control over how data persisted, enabling customization for complex scenarios like:

- Custom Table Structures: When data is stored in unconventional formats or across multiple tables.
- Advanced Integrations: When saving data involves external APIs or systems.
- Complex Logic: For operations requiring extensive calculations, validations, or dependencies between entities.

Steps to implement unmanage save.

Before that ensure persistence, table is not specified, as unmanaged save implementation cannot have persistence table specified to it.

Step 1: Include the **unmanaged save** keywords for all nodes where unmanaged behavior is required.

```
69 define behavior for zi_images
70 implementation in class zcl_b_images unique
71 //persistent table zimages_table
72 with unmanaged save
73 lock dependent by _saler
74 authorization dependent by _saler
75 early numbering
76 {
77     update;
78     delete;
79     field ( readonly ) ImageNumber, Matno, Salerid;
80     association _material;
81     ancestor association _saler;
```

Step 2: Redefine method **save_modified** in Saver Local class implementation level.

```
CLASS lhs_zi_saler DEFINITION INHERITING FROM cl_abap_behavior_saver.
    PROTECTED SECTION.
    METHODS : save_modified REDEFINITION.
ENDCLASS.

CLASS lhs_zi_saler IMPLEMENTATION.
    METHOD save_modified.
```


Tuesday, December 30, 2024

By: Devi Prasad Mishra

```
103 * -----
104 * Unmanaged Save for Image entity
105 * -----
106 * IF create-zi_images IS NOT INITIAL.
107 *   DATA : lt_images TYPE STANDARD TABLE OF zimages_table,
108 *         ls_return TYPE bapiret2.
109 *
110 *   lt_images = CORRESPONDING #( create-zi_images MAPPING image_number = ImageNumber
111 *                               attachment = ImageAttachment
112 *                               id = Id
113 *                               matno = Matno
114 *                               salerid = Salerid
115 *                               name = ImageName ).
116 *
117 *   CALL FUNCTION 'ZIMAGES_CUD' DESTINATION 'NONE'
118 *     EXPORTING
119 *       createx = 'X'
120 *       values  = lt_images
121 *     IMPORTING
122 *       return  = ls_return.
123 *   INSERT zimages_table FROM TABLE @lt_images.
124 *
125 *
126 * ENDIF.
127 *
128 * IF update-zi_images IS NOT INITIAL.
129 *   lt_images = CORRESPONDING #( update-zi_images MAPPING image_number = ImageNumber
130 *                               attachment = ImageAttachment
131 *                               id = Id
132 *                               matno = Matno
133 *                               salerid = Salerid
134 *                               name = ImageName ).
135 *   UPDATE zimages_table FROM TABLE @lt_images.
136 * ENDIF.
137 *
138 * IF delete-zi_images IS NOT INITIAL.
139 *   lt_images = CORRESPONDING #( delete-zi_images MAPPING image_number = ImageNumber
140 *                               matno = Matno
141 *                               salerid = Salerid ).
142 *
143 *   DELETE zimages_table FROM TABLE @lt_images.
144 * ENDIF.
145 *
146 * ENDMETHOD.
147 *
148 * ENDClass.
```

Here, I have used SQL queries, but instead, you can call external APIs or existing function modules to perform Create, Update, or Delete (CUD) operations.

Fiori Application Preview.

4149 /

phone

EditDeleteNot Available

Materials Information

Images Information

Materials Ids:
1

UnitField:
PC

Price:
329.00 AOA

Material:
phone

Quantity:
12 PC

Status:
A

Material Type:
Electronics (ELC)

Currencycode:
Angolanische Kwanza (AOA)

TotalPrice:
3,948.00 AOA

Images Information (3) Standard

CreateDelete

Attachment	ImageNumber	Material Number	
☐ download.jpg	1	1	>
☐ OnePlus 11R	2	1	>

Create

Image Information

ImageNumber:
3

Material Number:
1

Image Name:
OnePlus Nord

Images Information (3) Standard

Attachment	ImageNumber	Material Number	
☐ Oneplus 8T	1	1	>
☐ OnePlus 11R	2	1	>
☐ -	3	1	>

Update

Image Information

ImageNumber:
3

Material Number:
1

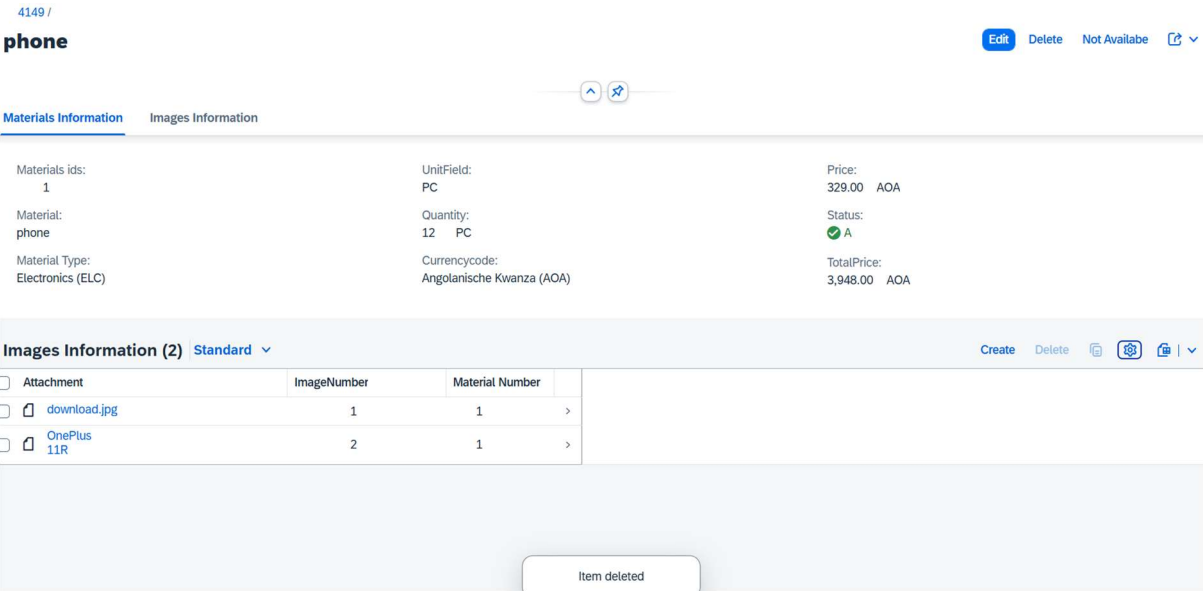
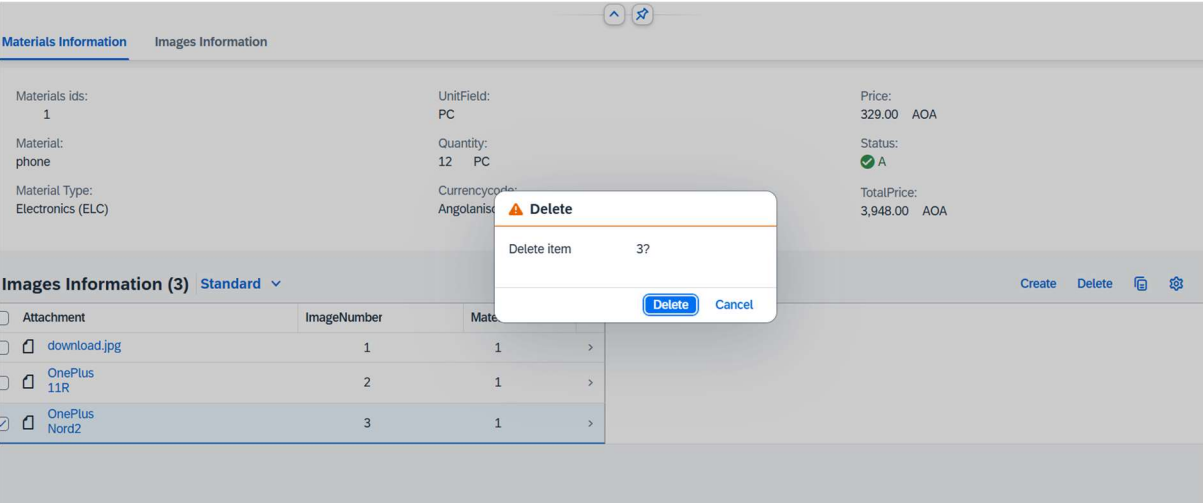
Image Name:
OnePlus Nord2

Attachment:
-

Images Information (3) Standard

Attachment	ImageNumber	Material Number	
☐ download.jpg	1	1	>
☐ OnePlus 11R	2	1	>
☐ OnePlus Nord2	3	1	>

Delete



Conclusion:

In RAP applications, both **Additional Save** and **Unmanaged Save** offer essential flexibility for customizing data persistence:

- **Additional Save** enhances the standard managed save sequence, allowing developers to introduce custom functionality (like logging or external integrations) while still utilizing the framework’s built-in features for validation and lifecycle management.
- **Unmanaged Save** provides full control over the save process, enabling developers to implement custom Create, Update, delete (CRUD) operations, such as using SQL queries, external APIs, or function modules, for more complex or non-standard requirements.

Both methods empower developers to tailor data management in the RAP framework, ensuring the application can meet diverse and complex business needs.